Week 1 - Domain 1: SDLC Automation - AWS CodeBuild

Study week: Oct 19 - Oct 25, 2026 **Exam:** AWS DOP-C02 (Domain 1 = 22% of the exam)

Covers: buildspec.yml phases, environment variables, Parameter Store / Secrets Manager, caching, VPC builds, reports

Part 2: AWS CodeBuild

2.1 What CodeBuild is

- **A fully managed build service.** Compiles code, runs tests, and produces artifacts or Docker images. No build servers (unlike Jenkins), scales automatically, **pay per build minute**.
- Each build runs in a **fresh, temporary container**. Nothing carries over between builds unless you use caching.
- **Build images:** AWS-managed images (Amazon Linux, Ubuntu, Windows, ARM, GPU), or **custom images** from ECR / Docker Hub when you need special tools.
- **Compute types:** small, medium, large, 2xlarge, plus Lambda compute for fast short builds. **Reserved capacity fleets** keep machines warm.
- **Timeouts:** build timeout defaults to **60 minutes**, maximum **8 hours**. A separate **queued timeout** limits how long a build waits in the queue.
- **Sources:** CodeCommit, GitHub, GitHub Enterprise, Bitbucket, GitLab, S3, or the CodePipeline artifact.
- **Starting builds automatically:**
  - **Webhooks** work for **GitHub, Bitbucket, and GitLab**, with **filter groups** (event `PULL_REQUEST_CREATED`, branch `^refs/heads/feature/.*`, file path).
  - **CodeCommit has no webhooks** - use an **EventBridge rule** or CodePipeline.

2.2 buildspec.yml: the build instructions

- Default location: `buildspec.yml` in the **root of the source code**.
- Override with a different file name or path (for example `buildspecs/test.yml`) or an inline buildspec - this is how you run **several build projects from one repository**.

```
version: 0.2

run-as: root                # optional: Linux user that runs the commands

env:
  shell: bash
  variables:                # plain-text values (NOT for secrets)
    APP_ENV: "staging"
  parameter-store:          # read from SSM Parameter Store
    DB_HOST: "/myapp/staging/db_host"
  secrets-manager:          # read from Secrets Manager
    DB_PASSWORD: "prod/myapp/db:password"
  exported-variables:       # pass values on to later CodePipeline stages
    - IMAGE_TAG
  git-credential-helper: yes

phases:
  install:
    runtime-versions:
      python: 3.12
      nodejs: 20
    commands:
      - pip install -r requirements.txt
  pre_build:
```

```
    commands:
      - aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin
    $ECR_URI
      - export IMAGE_TAG=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c 1-7)
  build:
    on-failure: ABORT        # ABORT (default) or CONTINUE
    commands:
      - pytest --junitxml=reports/junit.xml
      - docker build -t $ECR_URI:$IMAGE_TAG .
    finally:
      - echo "runs even if the commands above fail"
  post_build:
    commands:
      - |
        if [ "$CODEBUILD_BUILD_SUCCEEDING" = "1" ]; then
          docker push $ECR_URI:$IMAGE_TAG
        fi
      - printf '[{"name":"web","imageUri":"%s"}]' $ECR_URI:$IMAGE_TAG > imagedefinitions.json

reports:
  pytest-reports:
    files: ["junit.xml"]
    base-directory: reports
    file-format: JUNITXML
artifacts:
  files:
    - imagedefinitions.json
    - appspec.yml
    - "scripts/**/*"
  discard-paths: no

cache:
  paths:
    - "/root/.cache/pip/**/*"
    - "node_modules/**/*"
```

Phase order:

```
SUBMITTED > QUEUED > PROVISIONING > DOWNLOAD_SOURCE > INSTALL > PRE_BUILD
          > BUILD > POST_BUILD > UPLOAD_ARTIFACTS > FINALIZING > COMPLETED
```

Phase	What to put in it
install	Install tools, choose runtime-versions
pre_build	Log in to ECR, install dependencies, lint, set variables
build	Compile, run unit tests, docker build
post_build	Push the image, package output, send notifications

Behavior the exam likes to ask about:

- If **install or pre_build fails**, the build stops - build and post_build are skipped.
- If **build fails, post_build still runs.** Check `CODEBUILD_BUILD_SUCCEEDING` (1 = success so far, 0 = failure) so you don't push a broken image.
- A `finally` block runs even if the commands before it in that phase failed.
- `on-failure: CONTINUE` lets the build carry on to the next phase after a failure.

2.3 Environment variables

Precedence (highest first):

1. Values passed when the build is started (`start-build --environment-variables-override`, or the CodePipeline action configuration).
2. Values in the build project's environment settings.
3. Values in the buildspec `env` section.

Project variable types: `PLAINTEXT`, `PARAMETER_STORE`, `SECRETS_MANAGER`.

Exam trap: never put secrets in `PLAINTEXT` - plain-text values are visible in the console, API output, and CloudTrail.

Built-in variable	Meaning
CODEBUILD_RESOLVED_SOURCE_VERSION	The commit ID that was built (often the image tag)
CODEBUILD_SOURCE_VERSION	The branch, commit, PR, or S3 object version requested
CODEBUILD_BUILD_ID / CODEBUILD_BUILD_NUMBER	Unique build ID / increasing build number
CODEBUILD_BUILD_SUCCEEDING	1 or 0 - use it in post_build
CODEBUILD_WEBHOOK_TRIGGER	What started the build, e.g. pr/12 or branch/main
CODEBUILD_SRC_DIR	Where the source code is checked out
AWS_REGION, AWS_DEFAULT_REGION	The Region the build runs in

Passing values to later pipeline steps: `exported-variables` makes a value (e.g. `IMAGE_TAG`) available to later CodePipeline actions through a **variable namespace**, e.g. `#{BuildVariables.IMAGE_TAG}`.

2.4 Secrets: Parameter Store and Secrets Manager

```
env:
  parameter-store:
    API_URL: "/myapp/prod/api_url"            # String or SecureString
  secrets-manager:
    DB_PASS: "prod/db-creds:password"          # secret-id:json-key:version-stage:version-id
    API_KEY: "arn:aws:secretsmanager:us-east-1:111122223333:secret:apikey-AbCd"
```

The build's service role needs:

- `ssm:GetParameters` on the parameter.
- `secretsmanager:GetSecretValue` on the secret.
- `kms:Decrypt` - only if encrypted with a **customer-managed key**.

Other points:

- Values are **masked in build logs**.
- **Secrets Manager** = automatic **rotation** (e.g. RDS passwords) or **cross-account sharing**. **Parameter Store** = cheaper, for config values and simple secrets.
- To **pull a private build image** from Docker Hub, set the project's **registry credential** to a Secrets Manager secret. To run `docker login` inside the build, read the password from Secrets Manager into an environment variable.

2.5 Caching

Cache type	Where it lives	How it works	Use it for
Amazon S3 cache	S3 bucket	Downloaded at build start, uploaded at end. Lasts and is shared across hosts.	Dependencies when builds don't run often
Local - source cache	Build host	Keeps Git metadata so only changes are fetched	Large repositories
Local - Docker layer cache	Build host	Reuses Docker image layers. Requires privileged mode.	Docker image builds
Local - custom cache	Build host	Keeps folders listed under cache.paths	Fast builds that run often

- The **local cache is best-effort** - only helps when builds run often on the same host; not guaranteed.
- **S3 caching is reliable but slower** to restore.
- "Docker builds are slow": **local Docker layer cache**, `docker build --cache-from <ECR image>`, or a **custom build image** with tools preinstalled.
- **Building Docker images** inside CodeBuild needs **privileged mode**.

2.6 Builds inside a VPC

- **Why:** reach **private resources** - RDS, ElastiCache, internal APIs, private CodeArtifact / Nexus - usually for integration tests.
- **Setup:** in the project, choose the **VPC ID**, **private subnets**, and **security groups**.
- CodeBuild creates an **ENI** in your subnet with **no public IP**. Even in a public subnet it **cannot reach the internet**.
- Internet access (pip, npm, Docker Hub) needs a **NAT gateway** from a **private subnet**.
- Or avoid the internet with **VPC endpoints**:

Interface endpoints	Gateway endpoint
ecr.api, ecr.dkr, logs, ssm, secretsmanager, sts, codeartifact.api, codeartifact.repositories	S3 (also needed for ECR image layers)

- **Service role EC2 permissions:** `ec2:CreateNetworkInterface`, `ec2:DescribeNetworkInterfaces`, `ec2:DeleteNetworkInterface`, `ec2:DescribeSubnets`, `ec2:DescribeSecurityGroups`, `ec2:DescribeDhcpOptions`, `ec2:DescribeVpcs`, `ec2:CreateNetworkInterfacePermission`.

Classic exam scenario: "After setting the VPC configuration, builds fail to download dependencies or time out reaching S3." Fix: **private subnets with a NAT gateway**, or **VPC endpoints**. Moving to a public subnet does **not** fix it.

2.7 Test and code coverage reports

Report type	Supported formats
Test reports	JUnit XML, Cucumber JSON, TestNG XML, NUnit / NUnit3 XML, Visual Studio TRX
Code coverage	JaCoCo XML, Cobertura XML, Clover XML, SimpleCov JSON, LCOV

- **Report groups** collect results from each build and show pass/fail trends in the console.
- Configure in the buildspec `reports:` section, or point at an existing report group ARN.
- Report data is kept for **30 days**; **export raw files to S3** (optionally KMS-encrypted) to keep them longer.
- **Service role needs:** `codebuild:CreateReportGroup`, `codebuild:CreateReport`, `codebuild:UpdateReport`, `codebuild:BatchPutTestCases`, `codebuild:BatchPutCodeCoverages`.

2.8 Logging, monitoring, debugging, and batch builds

- **Logs:** CloudWatch Logs (default), S3, or both.
- **CloudWatch metrics:** `Builds`, `SucceededBuilds`, `FailedBuilds`, `Duration`, `QueuedDuration` - you can alarm on these.
- **EventBridge events:** "CodeBuild Build State Change" (IN_PROGRESS, SUCCEEDED, FAILED, STOPPED) and "Build Phase Change" - for notifications and automation.
- **Notification rules** send messages to SNS or Slack.
- **Debugging:** Session Manager into a running build (`codebuild-breakpoint` pauses it); the **CodeBuild local agent** runs a buildspec on your machine.
- **Batch builds:** `build-list` (parallel), `build-matrix` (multiple runtimes / OS / variables), `build-graph` (dependent builds).

2.9 Exam traps: quick table

Scenario	Answer
Database password visible in the console	Move it to a SECRETS_MANAGER / PARAMETER_STORE variable
Secret must rotate automatically	Secrets Manager
Build when a CodeCommit pull request is opened	EventBridge rule -> CodeBuild (no CodeCommit webhooks)
Build on a GitHub PR for feature/* only	Webhook with filter groups

Scenario	Answer
"Cannot connect to the Docker daemon"	Turn on privileged mode
Build in a VPC can't reach the internet	Private subnet with NAT gateway, or VPC endpoints
Slow builds - dependencies download every time	S3 cache or local custom cache
Pass the image tag to a later deploy stage	exported-variables + CodePipeline variable namespace
Don't push the image if tests failed	Check CODEBUILD_BUILD_SUCCEEDING in post_build
Test pass/fail trends over time	Report groups (JUnit, etc.)
Special tools baked into the build	Custom build image in ECR

2.10 Hands-on lab (about 2 hours)

1. Create a small Python app with `pytest` tests and push it to your CodeCommit repository.
2. Create a CodeBuild project: managed Amazon Linux image, **privileged mode** on, CloudWatch Logs on.
3. Store a fake secret `dop/lab/api:key` in Secrets Manager and a parameter `/dop/lab/env` in Parameter Store.
4. Write a buildspec that reads both values, runs `pytest --junitxml`, adds a `reports:` section, builds a Docker image, and pushes it to ECR tagged with the commit ID.
5. Run the build. Confirm the secret shows as `***` in the logs and the test report appears in the console.
6. Add an S3 cache for pip. Run twice and compare the install phase duration.
7. Break it on purpose: make a test fail (post_build runs but the image is not pushed); turn privileged mode off and read the Docker error.
8. Optional: put the build in a private subnet without a NAT gateway and watch it fail; then add NAT or endpoints.
9. Clean up: ECR images, NAT gateway (hourly cost), secret, project.

2.11 Check yourself

1. The build phase fails. Does post_build run? How do you stop the image from being pushed?
2. A CodeBuild project was moved into a VPC to reach RDS. Now `npm install` times out. Two possible fixes?
3. Which setting is required to run `docker build` inside CodeBuild?
4. How do you pass the Docker image tag from CodeBuild to a later deploy action in CodePipeline?
5. The same variable is set in the buildspec, the project, and at build start. Which wins?

Answers:

1. **Yes, post_build still runs.** Wrap the push in `if [ "$CODEBUILD_BUILD_SUCCEEDING" = "1" ]`.
2. **Private subnets with a NAT gateway**, or a **CodeArtifact npm proxy with VPC endpoints**. A public subnet will not work.
3. **Privileged mode.**
4. **exported-variables** with a namespace on the build action, then `#{Namespace.IMAGE_TAG}`. For ECS you can also write the image into `imagedefinitions.json`.
5. The value **passed at build start**, then the project, then the buildspec.

2.x More exam scenarios (added in the gap review)

- **Fixed outbound IP for a partner allow-list:** run the build in **private subnets** of a VPC that route through a **NAT gateway with an Elastic IP**. Elastic IPs can't be attached to CodeBuild, and public subnets don't work.
- **Builds wait 60-90 seconds in PROVISIONING:** use a **reserved capacity fleet** so machines are already running (or Lambda compute for small, fast builds).
- **Artifacts encrypted with a customer-managed key:** set the key as the **project encryption key**, and allow the service role in the key policy.